

OO y Tipos Abstractos de Datos

Programación Orientada a Objetos

1. Dado el siguiente código java, basado en la clase **Pair** definida en el teórico, explica qué sucede en memoria al ejecutar:

```
Pair p = new Pair(3, 5);
```

Explica brevemente ¿Qué diferencia hay entre clase y objeto?. Cuales son los atributos y métodos de **Pair**?

2. Completa la implementación de la clase **Persona** y **Estudiante** dadas en el teórico.
 - (a) Agrega al estudiante un atributo de tipo entero, privado, llamado **cantidadMateriasAprobadas** y además agrega un método **aproboMateria(int nota)** que actualice su promedio y la cantidad de materias aprobadas hasta el momento.
 - (b) En el main, crea un arreglo de tipo **Persona** que contenga tanto personas como estudiantes, y recorre el arreglo mostrando su información.
3. Prueba el ejemplo visto en el teórico de **Pair** y **MemoryPair**(como subclase de **Pair**). Escribe otro main donde pruebes el comportamiento de esta jerarquía.
4. Implementa una clase genérica **Caja<T>** con los siguientes requisitos:
 - (a) Define un atributo **contenido** de tipo **T**, el mismo debe ser privado.
 - (b) Implementa los siguientes métodos:
 - **void guardar(T item)**, el cual almacena un objeto en la caja.
 - **T obtener()**, el cual retorna el objeto almacenado.
 - **void mostrarInfo()**, el cual muestra información sobre el objeto almacenado por la terminal.
 - (c) Luego, en un main crea objetos del tipo:
 - **Caja<Integer>**
 - **Caja<String>**
 - **Caja<Persona>** (usando la clase **Persona** definida en 2)además el main debe realizar lo siguiente:
 - Usa **mostrarInfo()** para visualizar los contenidos de todas las cajas definidas.
 - Para cada caja definida recupera el objeto almacenado en la misma, usando **obtener()**, y muestra sus atributos si es un objeto de tipo **Persona**.
 - Crea una lista de cajas genéricas (**ArrayList<Caja<?>>**) y muestra el contenido de cada una.
5. Define una clase abstracta **Figura** con métodos abstractos **area()** y **perimetro()**. Luego :
 - (a) Implementa las subclases **Circulo**, **Rectangulo** y **Triangulo**.
 - (b) En un main, crea un **ArrayList<Figura>** y calcula el área total.
6. Define una clase abstracta **Empleado**, con atributos **nombre** y **salarioTotal**. Luego implementa subclases:
 - (a) **EmpleadoPorHora** (con atributos **horasTrabajadas** y **tarifaPorHora**)
 - (b) **EmpleadoMensual**

Implementa el método **calcularPago()** en cada subclase.

Finalmente, en un main, crea una lista de empleados y muestra en la terminal el nombre y salario de cada uno, usando polimorfismo.

Uso e implementación de TADS

1. Explica con tus palabras la diferencia entre **tipo de dato**, **estructura de datos** y **tipo abstracto de dato** (TAD). Da un ejemplo en Java para cada uno.
2. Implementa una función en Java utilizando la clase `LinkedList` que cree una lista de N números enteros aleatorios y la retorne como resultado. N deber ser un parámetro de la función. Luego, en un main, invoca dicha función e imprime la lista generada.
Hint: Investiga cómo se utiliza la clase `java.util.Random`, para generar números aleatorios.
3. Implementa una función `pertenece` en Java, que dada una lista de N números enteros y un número, retorne verdadero si el valor recibido como parámetro se encuentra en la lista, falso en caso contrario.
4. Investiga en que consiste el método de ordenamiento denominado `BubbleSort` u ordenamiento `Burbuja`. Describe paso a paso como ordenaria este método el siguiente arreglo: `[7, 9, 3, 1]`.
5. Define en Java una función que implemente el método `BubbleSort` sobre una lista de enteros utilizando la clase `ArrayList`.
6. Define una interfaz genérica `Lista<T>` con las siguientes operaciones:

- agregar un elemento:
 - `void insertarFin(T item)`
 - `void insertarInicio(T elem)`
 - `void insertarPos(T elem, int pos)`
- eliminar un elemento:
 - `T eliminarUltimo()`: Elimina y devuelve el último elemento de la lista.
 - `T eliminarPrimero()`: Elimina y devuelve el primer elemento de la lista.
 - `T eliminarPos(int pos)`: Elimina el elemento en la posición especificada.
- obtener un elemento (sin modificar la lista):
 - `T obtenerPrimero()` (precondición: La lista no es Vacía.)
 - `T obtenerUltimo()` (precondición: La lista no es Vacía.)
 - `T obtenerElemPos(int pos)`, (precondición: pos es una posición válida)
- otros métodos auxiliares:
 - `int buscar(T item)`: Retorna la posición en la que se encuentra un elemento o -1 en caso que no se encuentre.
 - `int longitud()` : Retorna la cantidad de elementos que actualmente almacena la lista.
 - `boolean esVacía()` : Retorna la verdadero si la lista no contiene elementos, falso en caso contrario.
 - `void imprimir()`: Imprime por pantalla los elementos de la lista.

Crea dos clases que implementen esta interfaz:

- (a) `ListaEnlazada` : La cual implemente el TAD Lista utilizando memoria dinámica.
 - (b) `ListaArreglo` : La cual implemente el TAD Lista utilizando arreglos (memoria estática).
7. Importa la clase `Stack` de Java (`import java.util.Stack`) y utilízala para implementar un programa que dada una secuencia de caracteres, que representa una expresión matemática, verifique si la misma tiene los paréntesis correctamente balanceados.

- `"(3+4) * 8"` está balanceada.
 - `"(( 3 + 4 ) * 8 "` no está balanceada.
 - `" ) 3 + 4 ( * 8 "` no está balanceada, ya que si bien cada paréntesis de apertura tiene un paréntesis de cierre, no están en el orden correcto.
8. Define una interfaz genérica `Pila`, con sus operaciones características.
Para unificar notación sugerimos que utilicen los siguientes nombres:
- `void apilar(T item)`
 - `void desapilar()`
 - `T tope()`
 - `boolean esVacia()`
 - `void vaciar()`
9. Crea dos clases que implementen la interfaz `Pila`, definida anteriormente en 8, utilizando las clases `ListaEnlazada` y `ListaArreglo` implementadas en el ejercicio 6.
10. Reimplementa el ejercicio 7 utilizando cada una de las clases de Pilas implementadas en el ejercicio 8. Verifica que el programa funciona cómo se espera.
11. Define una interfaz `Cola` genérica con las operaciones características de este TAD. Crea dos clases que implementen esta interfaz, utilizando las clases `ListaEnlazada` y `ListaArreglo` implementadas en el ejercicio 6.
Para unificar notación sugerimos que utilicen los siguientes nombres:
- `void encolar(T item)`
 - `void desencolar()`
 - `boolean esVacia()`
 - `void vaciar()`
12. Utilizando alguna de las implementaciones del TAD Cola, simule una cola de trabajos de impresión donde se agregan y procesan trabajos en el orden en que llegan. Para ello vamos a simplificar la representación de los trabajos a imprimir en un string que representará el nombre del archivo a procesar:
- ```
Cola<String> colaImpresion = new Cola<>();

colaImpresion.encolar("Trabajo1.txt");
colaImpresion.encolar("Trabajo2.pdf");
colaImpresion.encolar("Trabajo3.doc");
```
13. Averigua cómo se implementan las colas con arreglos circulares. Qué ventajas presenta esta implementación?
14. Realiza una implementación en Java de una Cola Circular para enteros, con las siguientes operaciones:
- `boolean estaLlena()`
  - `boolean estaVacia()`
  - `void encolar(int elemento)`
  - `void desencolar()`
  - `void imprimir()`
15. Implemente, utilizando una pila, una función que dado un string que representa una expresión en notación postfija, evalúe la misma y retorne el resultado.
- (a) `"74 - 63 + *"`, resultado esperado : 27
  - (b) `"512 + 4 * +3 -"`, resultado esperado : 14
  - (c) `"392 + *8 - 5 /"`, resultado esperado : 5

**Hint:** Investigue la clase `String`, y los métodos disponibles para dividir una cadena.